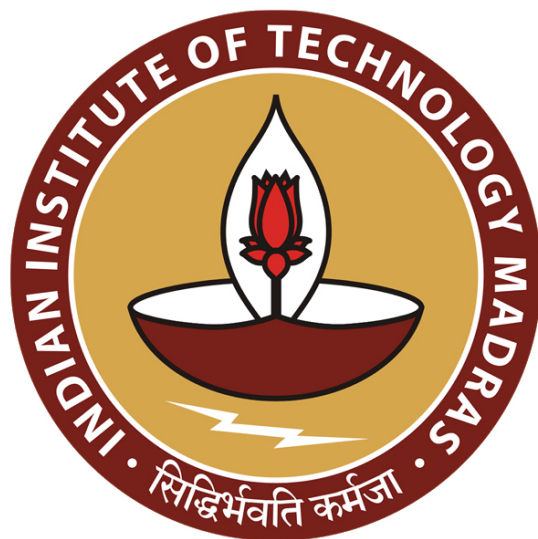




IIT Madras
ONLINE DEGREE

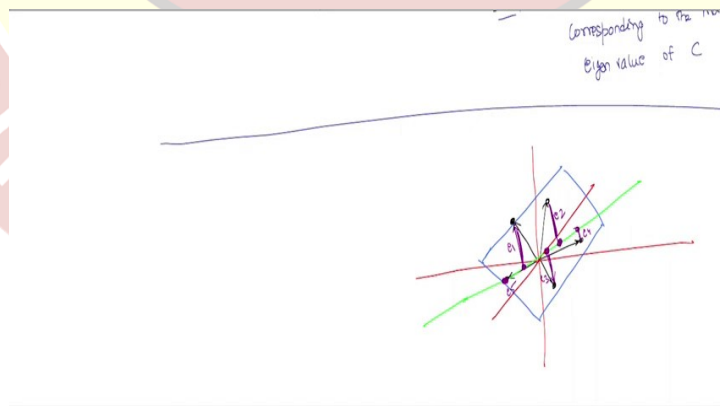
Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science and Engineering
Indian Institute of Technology, Madras
Representation Learning: Part 3

Even if you assume that we can solve this problem, the more pertinent question that we should ask ourselves is that are we done? In other words, let us take a step back and think about where we started. So we were given a data set and we asked for that, on which we can find proxies for the data points such that once you find the proxies you can compress the data using just a representative and coefficients along this line.

Now how do we do that? Well, we are saying we can set up an optimization problem and solve it and it turns out also that solution to the optimization problem also turns out to be the eigenvector of a nice matrix called the covariance matrix associated with the data set; that is incidental at this point.

So, but the more important question is let us say we think of this solution to this optimization problem as a black box, we give the data set and then the line comes out, we know what the black box did now, but let us say for the moment we think of it as a black box. Now are we satisfied is the question we should ask, so are we satisfied with this compression that we have in all cases? Well not necessarily; to give an example let us consider the following picture.

(Refer Slide Time: 1:38)



So from 2D let us say we go to 3D, now so now you you are in three dimensions and let us say your data points are bunch of vectors in 3D, so let us say there are a bunch of vectors

here, here, here and so on here. Now imagine that all these vectors were along a plane, so it is a three dimensional space but then not all data points are scattered everywhere around, they all fall along a plane.

Now that is the critical structure that we have in this dataset, the question is we able to, in some sense extract that structure, is our compression trying to extract the structure or not? Well, what is our compression method trying to do? It is trying to find the best line on which you can project the data point such that you lose the least. For this particular data set, well, if we run our algorithm we are going to get a line.

So that line might be something like this; maybe this is the line. So this line will be necessarily on the plane, I mean, you can think about why it has to be on the plane, because if it is not on the plane, if it is outside the plane then it will cost more to project on something outside the plane. So you can avoid that cost. So necessarily the best line is going to be on the plane because all the data points are on the plane.

But where these proxies are going to be, well, these proxies are going to be along this line. So I am going to think of all the data points which were originally on the plane using their proxies along this line. Now the question is, are we satisfied with this? So is this a good compression? Well, it is a compression, if you want to compress this definitely this is a good way to do it, but then does it capture the necessary structure in the data?

In other words what I am saying is the following, our hypothesis is that there is a line that best represents this data, everything else that is the part that we think of as error, well, the part that we leave out while reconstruction is something that we are going to think of as error, but now if the data points lied along a plane, then this part which we are imagining as error may not necessarily be error, that might also contain some information.

So because necessary information, the structure is in a plane not on a line, so the bits that we lose necessarily in this particular case will also contain some information and we might want to extract that information out also, but then we have a nice black box algorithm, which will give you a good line. So we do not want to lose the power of our algorithm that we already have, at the same time we also do not want to just be satisfied with a single line.

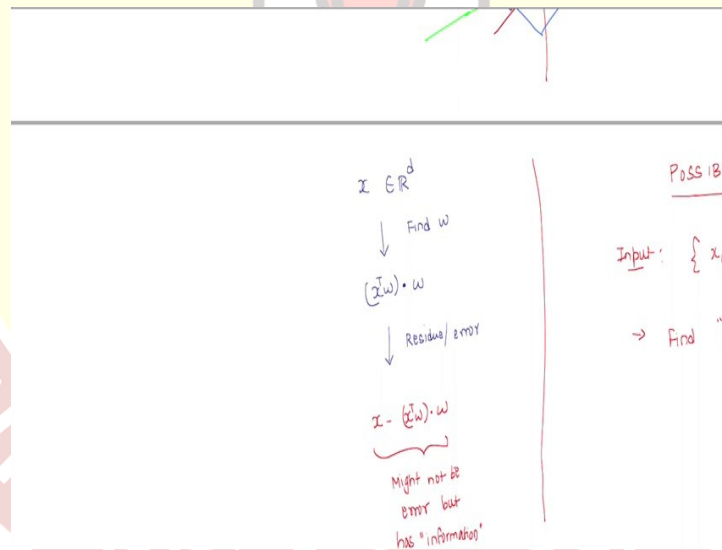
We want to somehow capture the fact that the data might be lying on a two dimensional plane on three dimensional data. So what can we do now? You can do one thing. So if you observe

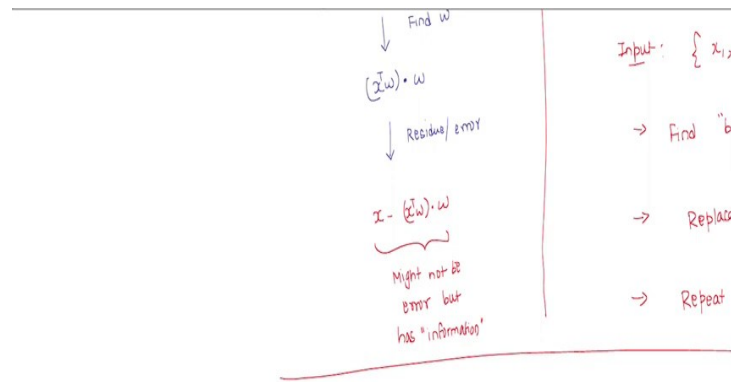
this, so now what we are seeing is that these are the proxies the purple points are the proxies and the purple vectors are the error vectors.

Now if you observe this what we are saying is that the error vectors are not necessarily error, that might also have information, which means that, for example, if I just plotted this error vectors on three dimensional space again, how would these error vectors look like? Now you can see already in this picture that all these error vectors will necessarily align in some direction. So this would be the first error vector corresponding to let us say e_1, e_2, e_3, e_4, e_5 .

e_5 is just the origin in this case, because x_5 point was already on the line. This could be e_1 , this could be e_3 , this could be e_4 , maybe this is e_2 , and so on. Now as you can see, all these so called error vectors also lie along a line, which means that there is some notion of information in the error vector also and we would want to extract that as well. Now how can we do that?

(Refer Slide Time 06:33)





One procedure that you could follow is the following. Now what we have done so far is we have a data point x , which is a d dimensional data point and what we are seeing is that we will find some w for the data set and once the w is found we are going to represent this data point as its proxy, which is $(x^T w)w$.

This is the representative w and $x^T w$ is the coefficient corresponding to this data point. But now the error or the residue or what we might, we have been calling as error that is just the remaining piece. So that is just $x - (x^T w)w$. Now what we are saying is that this might not be error, but might have information, so but has information.

Well, for one if all of this were error they would not necessarily line up along the same direction. So if it is error, the error for each data point has to be kind of in all over the place, so that is the high level idea. So now how can we work with this well we could do the following, so here is a possible algorithm and then we will kind of build this algorithm as we go along. A possible algorithm could be the following.

You start with your original data set which is your input $\{x_1, x_2, \dots, x_n\}$, where each x_i is in $\mathbb{R}^d$. The first step is whatever we have been doing so far find the best line w by minimizing the error or maximizing the w^T , does not really matter how we do this. For now let us call this w_1 instead of w , we will see why I am calling it w_1 . Now, what am I going to do? I am going to replace x_1 , the data point x_1 with its residue.

So the residue is this vector $x_1 - (x_1^T w)w$, so now I have a new data set, so now all my the data set, the second level data set is the data set that only contains the residue vectors with

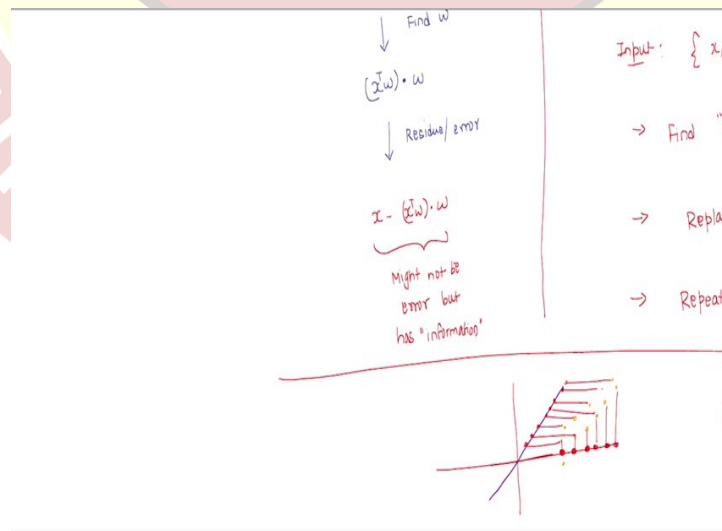
respect to the first line that I have found. Now, what I can do is I can repeat this procedure to obtain w_2 , the second level.

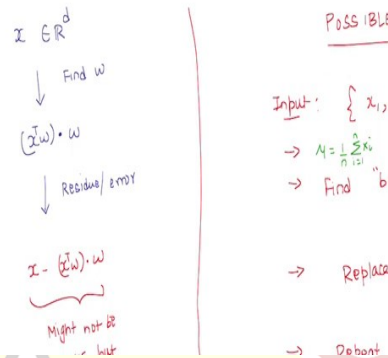
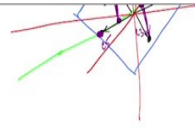
Why should I stop after getting the second level w_2 ? So the idea was the residue might contain information. Let us say now we had thousand dimensional data so we fit the best line, we found the best line w_1 we looked at the residue, well, let us say we think that the residue also has information, so we compute the residue data set again we find the second line and we could go on and on.

So we can go until, I mean up to the number of dimensions that you have; so you can add more and more data sets create more and more data sets using the residues and we can keep doing this. Now the question is, well, this is a possible algorithm but there are several questions that come up here. So one is, how do we do the reconstruction once we have these different w 's that we have found?

What does it mean to say that we have compressed here? So how do we reconstruct? What kind of compression are we achieving? This is an important question. We will answer that, but even before that another question that might arise is a more basic question and we will answer that and then we will answer the more pertinent question about reconstruction and so on.

(Refer Slide Time 10:54)





Another question that might arise is that the data itself may look something like this. So you might have data which is like this even for one dimension, so even for two dimensional data you might have data like this. Now remember, we are only finding lines that pass through the origin, because our notion of compression is representative and a coefficient multiplied by this representative, which means if the coefficient is 0, then the origin should be part of the line that we are considering.

So what might happen is that our data may not necessarily be around the origin, which means though the data is along the direction pointed by the purple line here, but if you really try to find proxies like the way we did, by looking at proxies like this, the errors might be much more than just projecting all the data points along the y-axis.

So if you find proxies along the y-axis, so it might so happen that well the y-axis is, sorry the x-axis, the x-axis in this case might be a better line in terms of the reconstruction than your actual line which you really want to get, which is capturing the direction in which the data is dispersed. So what can we do here? Well, one thing we can do is to recognize the issue first, the issue is that data may not be centered.

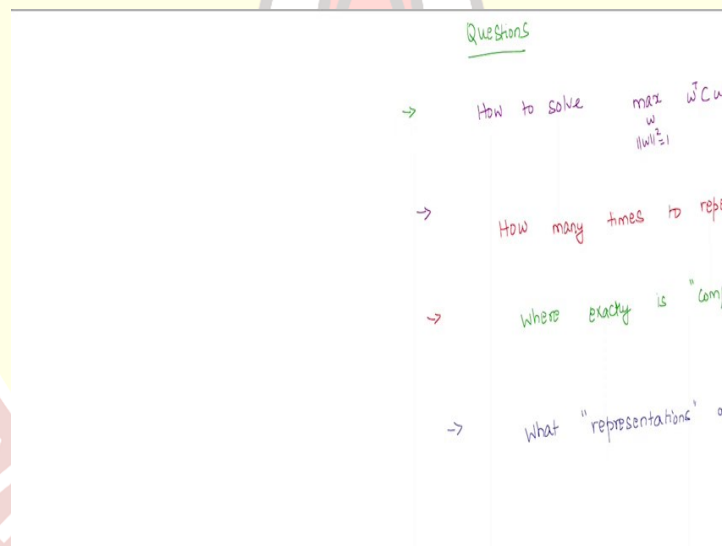
One small fix that you can do is first do a centering of the data set in this possible algorithm that we have, which is first bringing your data such that the center or the origin lies at the center of the data, such that when you search for lines that pass through the origin then you are necessarily finding the line which aligns with our intuition of the direction in which the data itself is dispersed.

So how could you do the centering? Well, the centering is simple easy, so you first find the mean of your data points which is the average of your data set given to you and then you first make your data set centered by subtracting the mean from your data points. So for all x_i , for all i . Now what does that mean?

Well, you are separating the mean from each of the data points, so now if you recompute the mean of this subtracted data set that would be the origin by definition because the mean itself is just the average of your data points, essentially you are moving your data points, translating your data points such that the origin is the mean.

So that that takes care of the centering issue, but still our previous issues are still pertinent. So several issues, so I am going to write down these issues and then we will try to answer each of these in the coming videos.

(Refer Slide Time 14:18)



So the questions that I am going to leave ourselves with right now are the following; of course, the first question is how to solve max of $w^T C w$ or $\|w\|^2=1$ like how I have been using, $w^T C w$, of course, I did hint at the answer to this that this will be an eigen vector solution, but we have to think about that a little bit more carefully.

The second thing is that, well, in this possible algorithm so how many times should we repeat the procedure, procedure of computing the residues and then finding the line that best fits the residues, repeat the procedure. Perhaps more importantly is the third point, where exactly is the compression happening now in the algorithm.

Earlier it was clear that we were finding one single line, we were finding proxies along this line and we knew that how the compression was playing out. So where exactly, but now, perhaps not that obvious where is the compression really happening. And the fourth point is that well earlier we know that it is a representative and a coefficient and then representative multiplied by this coefficient will give us the proxy.

Now we have multiple lines, perhaps multiple coefficients, so the question is what are the representations that we are really learning using this. So these are some questions that we need to answer to understand our procedure better and we will see that all of this will tie together nicely to whatever you may have learned in your linear algebra classes already in a Machine Learning Foundations' course. If not you this might give you still a different way of thinking about some of the algorithms that you may have already seen.

So if you answer all these four questions carefully then that will lead us to a very, very solid algorithm in data science, which will be our first algorithm for this course, which is a Representation Learning Algorithm for Unsupervised Learning. We will talk about what this algorithm is once we answer all these four questions.

And think about these four questions to see if you already have some ideas about what are the answers for these. So I will leave you with these questions for this point and we will come back and answer all these questions and develop the algorithm that we set out to in the next video. Thank you.